CNN và ResNet

Từ nét cục bộ đến nhãn của ảnh, từ 2 lớp đến 50 lớp

Phase 5 · Bài 5.2

Cần một nhãn cho cả ảnh

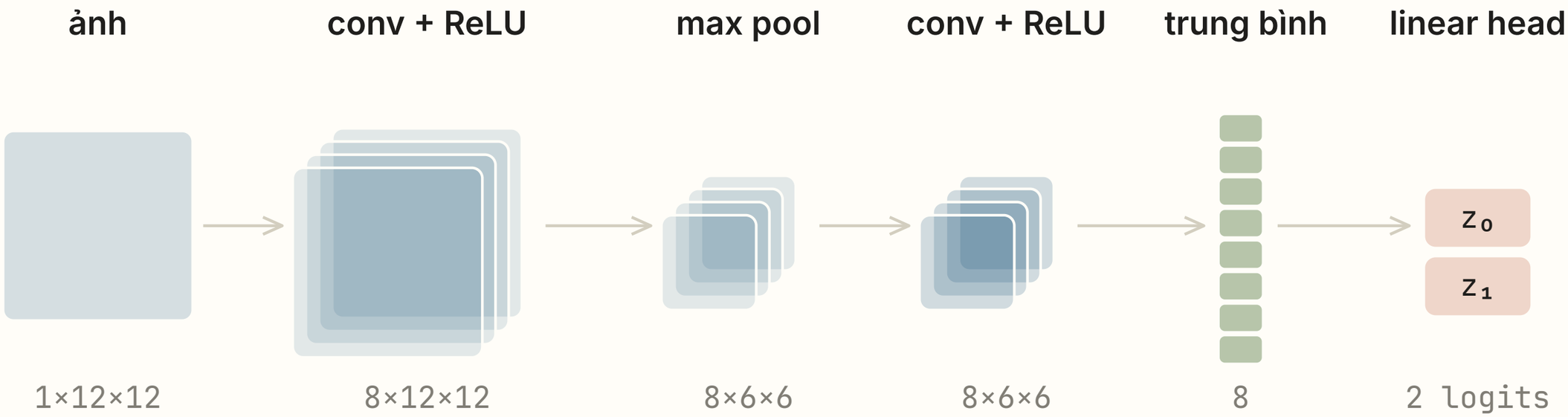
Một ảnh có thể chứa nét ngang, nét đứng và nhiễu.

Convolution tạo feature map, còn head gộp feature thành nhãn của cả ảnh.

01

Xây CNN nhỏ

Theo dõi shape



TinyCNN theo shape: hai lần conv + ReLU xen max pool, trung bình mỗi feature map thành một số, rồi linear head ra hai logits.

Activation và head

Thiếu ReLU, chồng conv vẫn chỉ là một phép affine.

Global average pooling biến mỗi kênh thành một số.

Head luôn nhận 8 số, dù ảnh lớn hay nhỏ.

Hai logits cho hai lớp; cross-entropy nhận logits trực tiếp.

HỎI NHANH

Head trả về logits `(N, 2)`. Nhãn đưa vào `CrossEntropyLoss` cùng logits ấy phải có shape và kiểu gì?

A `(N, 2)`, one-hot kiểu `torch.long`

B `(N, 1)`, float chứa 0.0 hoặc 1.0

C `(N,)`, kiểu `torch.long`, chứa chỉ số lớp

D `(N, 1, 12, 12)`, mask nhị phân theo từng pixel

Ảnh tự sinh để kiểm cơ chế

```
train_images, train_labels = make_lines(80, 1)
valid_images, valid_labels = make_lines(40, 2)
print(tuple(train_images.shape), torch.bincount(train_labels).tolist())
print(tuple(valid_images.shape), torch.bincount(valid_labels).tolist())
```

KẾT QUẢ

```
(80, 1, 12, 12) [40, 40]
(40, 1, 12, 12) [20, 20]
```

Tám ảnh train đầu tiên

8 ảnh train đầu tiên của make_lines (12×12, nhiễu σ = 0.04)



đứng ngang đứng ngang đứng ngang đứng ngang

chỉ 5 vị trí × 2 hướng: validation khác train ở nhiễu

Tám ảnh train đầu tiên do make_lines sinh (dữ liệu từ code bài): nét đứng và nét ngang dài 8 ô, rộng 2 ô, ở một trong 5 vị trí, cộng nhiễu nhỏ.

HỎI NHANH

`make_lines` chọn vị trí nét bằng `rng.integers(3, 8)`. Bỏ nhiễu đi thì có bao nhiêu ảnh sạch khác nhau, và validation kiểm được điều gì?

A 5 ảnh, validation kiểm cả vị trí lẫn hướng nét mới

B 10 ảnh, validation chỉ khác train ở phần nhiễu

C 10 ảnh, validation có những vị trí mà train chưa gặp

D 80 ảnh, vì mỗi ảnh train là một mẫu riêng

Kiểm trước khi train

`TinyCNN` và `train_images` đã được định nghĩa ở các khối trước. Kiểm batch trước khi chạy vòng train.

```
model = TinyCNN()
print(tuple(model(train_images[:2]).shape))
print("parameters:", sum(p.numel() for p in model.parameters()))
```

KẾT QUẢ

```
(2, 2)
parameters: 682
```

Đếm tham số

tham số.

; ; . Mỗi số là bias

của một kênh ra.

HỎI NHANH

Tensor trước head có shape `(8, 16, 4, 4)`. Gọi `flatten(0)` thay cho `flatten(1)` thì chuyện gì xảy ra?

A Cả batch bị gộp thành một vector `(2048,)`

B Chiều kênh và chiều batch đổi chỗ, ra `(16, 128)`

C Chỉ hai chiều không gian bị gộp, ra `(8, 16, 16)`

D Không đổi gì, vì PyTorch tự bỏ qua chiều batch

HỎI NHANH

Đổi conv thứ hai thành `Conv2d(8, 16, 3, padding=1)` và head thành `Linear(16, 2)`. TinyCNN mới có bao nhiêu tham số?

A 682

B 1264

C 1266

D 1282

02

Huấn luyện và đọc kết quả

Một bước cập nhật

Mỗi bước dùng cả 80 ảnh; `criterion` là `CrossEntropyLoss`, `optimizer` là Adam.

```
for step in range(100):
    model.train()
    optimizer.zero_grad()
    logits = model(train_images)
    loss = criterion(logits, train_labels)
    loss.backward()
    optimizer.step()
```

Kết quả

Mỗi cặp là `(loss, accuracy)`; loss đã làm tròn bốn chữ số.

KẾT QUẢ

```
before: (0.6981, 0.5)
train: (0.0, 1.0)
valid: (0.0, 1.0)
```

Validation từ 0.5, mức đoán bừa, lên 1.0.

Kết luận chỉ trong quy tắc sinh: 10 ảnh sạch cộng nhiễu.

HỎI NHANH

Đọc thấy `train: (0.0, 1.0)` và `valid: (0.0, 1.0)`, một bạn kết luận “model overfit, vì loss train đã bằng 0”. Nhận xét nào đúng?

A Đúng, loss train về 0 là dấu hiệu học thuộc

B Sai, overfit cần train tốt hơn validation một cách rõ rệt

C Đúng, vì mọi xác suất dự đoán đều bằng 1

D Sai, vì loss bằng 0 nghĩa là model đúng trên mọi ảnh

CNN đã nhìn thấy gì?

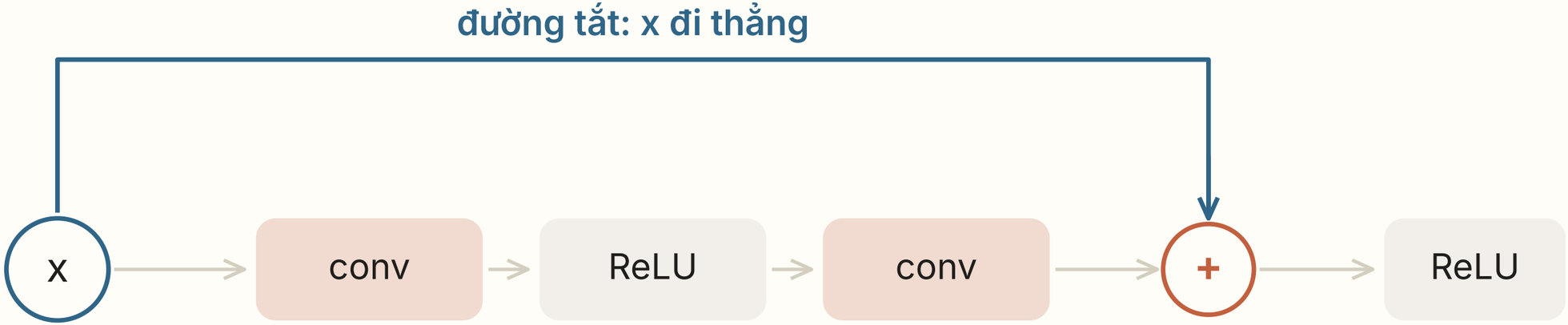


Hai ảnh validation và ba kênh thật sau Conv + ReLU đầu tiên của TinyCNN đã train (kênh có phương sai lớn nhất): mỗi kênh là một bộ dò riêng, nhấn những phần khác nhau của nét.

03

Giữ đường truyền thông tin

Học phần cần sửa



$$y = \mathrm{ReLU}(x + F(x))$$

Residual block của bài: nhánh F(x) gồm conv, ReLU, conv; đường tắt mang x cộng thẳng vào F(x) rồi mới qua ReLU.

Gradient có đường tắt

Tích qua nhiều block luôn chứa số hạng : một lối đi không qua nhánh nào.

HỎI NHANH

Mười block một chiều số , không có ReLU, block nào cũng có . Gradient đi qua cả mười block được nhân với bao nhiêu khi , và khi ?

A và

B Đúng bằng ở cả hai trường hợp

C Khoảng và khoảng

D ở cả hai trường hợp

Ví dụ tính tay

Có ReLU sau phép cộng: . Phải nói rõ ReLU đặt ở đâu.

Kiểm nhánh bằng 0

KẾT QUẢ

```
True
[[2.0, 0.0]]
```

: dữ liệu dương đi qua nguyên vẹn, nhưng ReLU sau phép cộng vẫn cắt thành .

Đổi kênh hoặc stride thì đường tắt cần projection để khớp shape.

HỎI NHANH

Nhánh chính cho ra `(N, 16, 3, 3)`, còn đường tắt là `(N, 8, 6, 6)`. Làm cách nào để cộng được, mà vẫn đúng nghĩa?

A Để broadcasting tự kéo đường tắt cho khớp

B `reshape` đường tắt thành `(N, 16, 3, 3)`

C Bỏ đường tắt ở block này cho khỏi lệch

D Dùng conv stride 2 đưa từ 8 lên 16 kênh

04

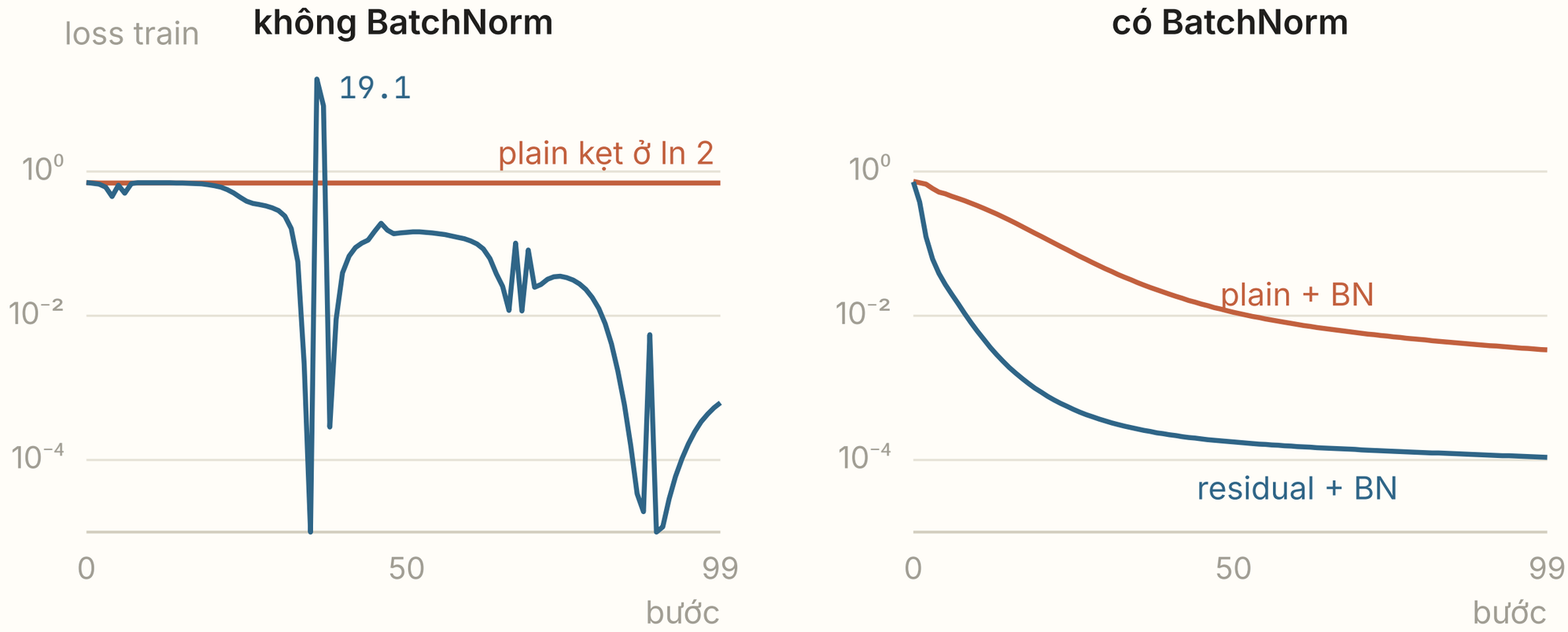
Mạng 18 lớp có và không có đường tắt

Bốn tổ hợp, mỗi lần một công tắc

8 block `DeepBlock(skip, norm)` sau conv đầu; cùng seed, Adam, 100 bước.

	grad	loss@10	max	loss@99	valid
plain	2.5e-11	0.6935	0.6980	0.6931	0.5
plain + BN	4.1e-01	0.3354	0.7324	0.0034	1.0
residual	3.1e-02	0.7010	19.1213	0.0006	1.0
residual + BN	2.6e-01	0.0061	0.7157	0.0001	1.0

Loss theo bước



Loss train theo bước của bốn mạng 18 lớp (số từ code bài, trục log): không BatchNorm, plain kẹt ở ln 2 còn residual học nhưng có bước vọt lên 19.1; có BatchNorm, residual hội tụ nhanh hơn plain.

Đọc bảng

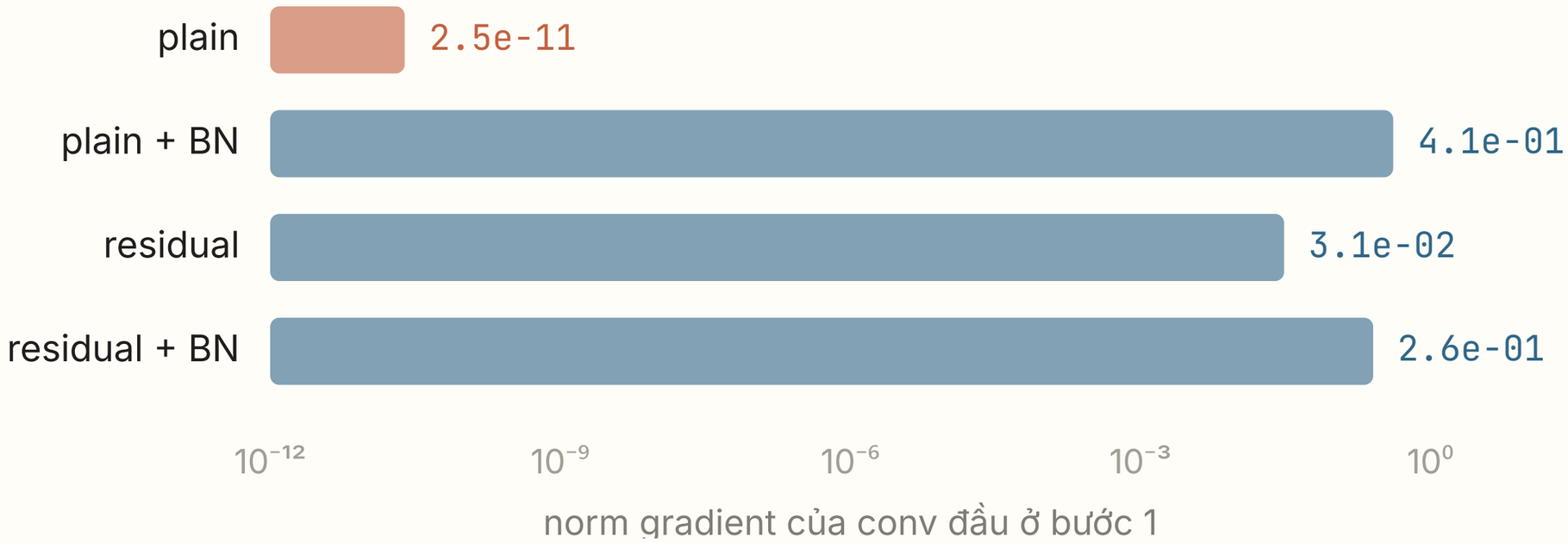
Plain không BN: gradient , kẹt ở (như bài 3.3).

Residual không BN: gradient lớn hơn lần nhờ số hạng ; học được nhưng có bước loss 19.1213.

Có BN: cả hai học; residual nhanh hơn hẳn (0.0061 so với 0.3354 sau 10 bước).

ResNet = đường tắt + BatchNorm.

Gradient tới conv đầu



Norm gradient tới conv đầu ở bước đầu tiên (số từ code bài, trục log): mạng plain không BatchNorm chỉ còn cỡ 10^{-11}; thêm đường tắt hoặc BatchNorm đưa nó lên cỡ 10^{-2} đến 10^{-1}.

HỎI NHANH

Theo bốn dòng output, thí nghiệm này hỗ trợ kết luận nào?

A Riêng đường tắt đã đủ để train ổn định

B Đã có BatchNorm thì đường tắt trở nên thừa

C Cùng có BatchNorm, mạng có đường tắt hội tụ nhanh hơn

D Mạng thường kẹt ở 0.6931 vì học thuộc tập train

Ở quy mô ImageNet

Plain 18 lớp và ResNet-18 chính xác gần nhau; ResNet-18 hội tụ nhanh hơn.

Plain 34 lớp có lỗi train cao hơn plain 18 lớp: khó tối ưu, không phải overfit.

05

ResNet thật

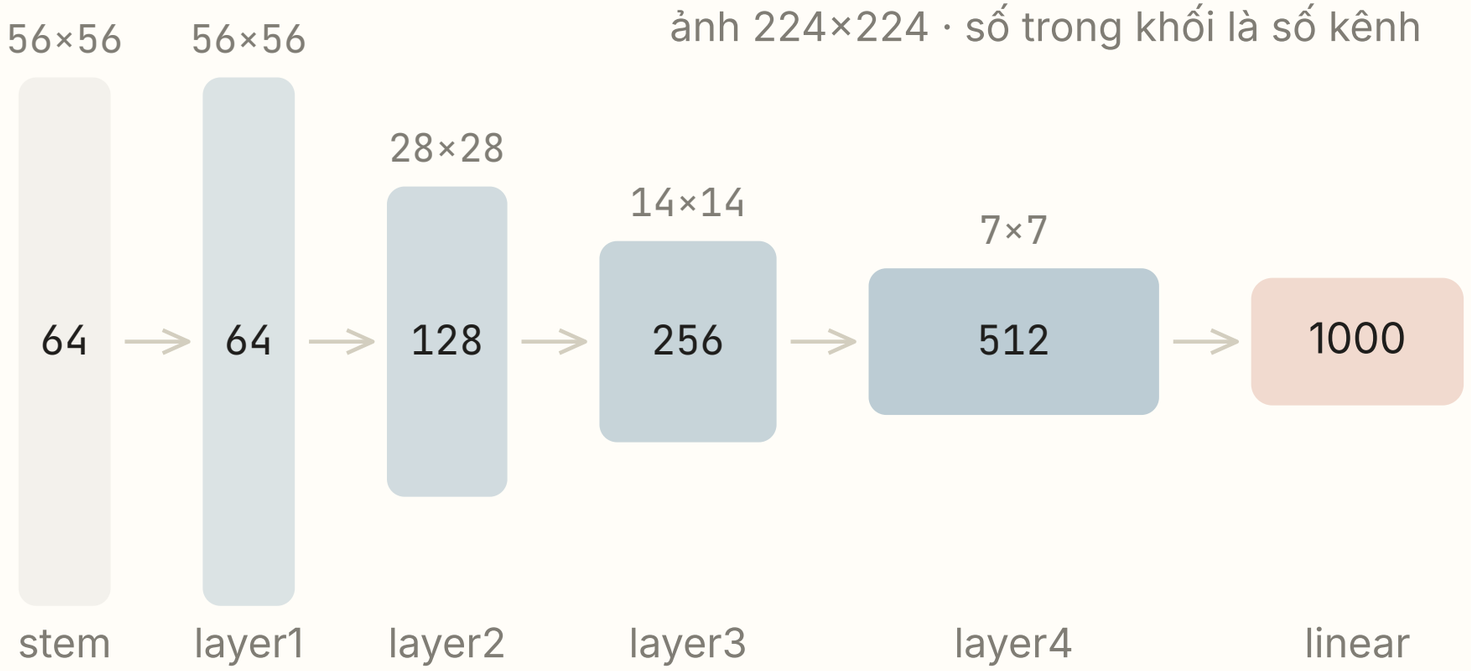
ResNet-18 qua từng stage

KẾT QUẢ

```
stem   (1, 64, 56, 56)
layer1 (1, 64, 56, 56)
layer2 (1, 128, 28, 28)
layer3 (1, 256, 14, 14)
layer4 (1, 512, 7, 7)
Conv2d(64, 128, kernel_size=(1, 1), stride=(2, 2), bias=False)
```

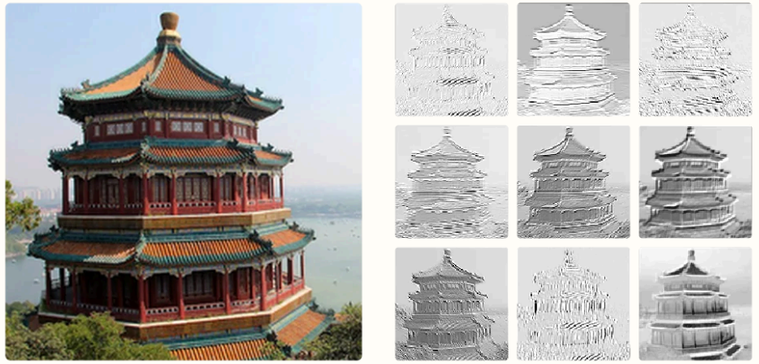
Cạnh giảm 32 lần; block đầu mỗi stage có projection stride 2.

Cạnh giảm, kênh tăng

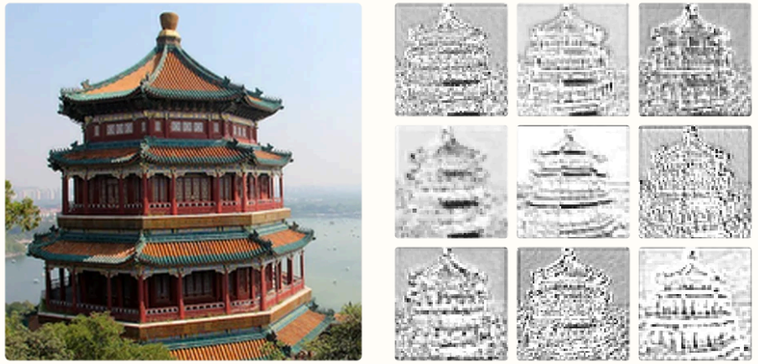


ResNet-18 theo stage (shape từ code bài): stem đưa ảnh 224×224 về 56×56; mỗi stage sau chia đôi cạnh và nhân đôi số kênh, tới 512 kênh 7×7 trước global pooling và lớp linear.

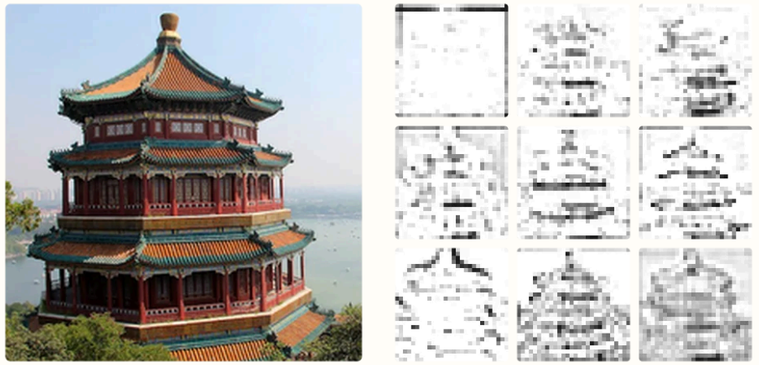
Feature map thật qua từng stage



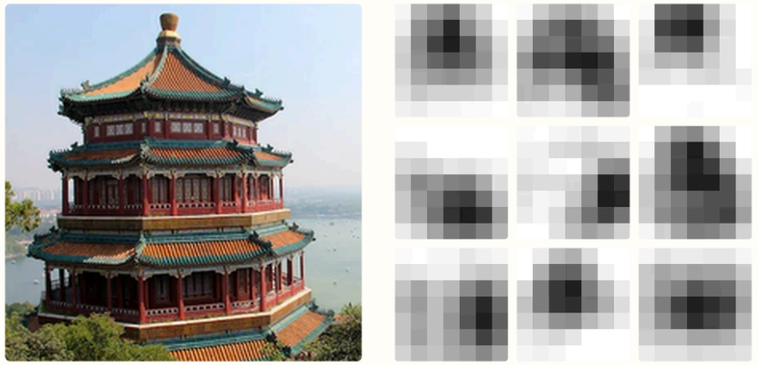
`conv1` · 64 kênh · 112 × 112 (9 kênh mạnh nhất)



`layer1` · 64 kênh · 56 × 56 (9 kênh mạnh nhất)



`layer2` · 128 kênh · 28 × 28 (9 kênh mạnh nhất)



`layer4` · 512 kênh · 7 × 7 (9 kênh mạnh nhất); dự đoán “bell cote” 0.54

Feature map thật của ResNet-18 trên ảnh tháp: càng sâu, bảng càng nhỏ và nét càng trừu tượng.

Bottleneck

Hai conv ở 256 kênh: , gấp gần 17 lần.

ResNet-50: khoảng 25.6 triệu tham số, gấp 2.2 lần ResNet-18.

Bottleneck so với hai conv

bottleneck

1×1 256 → 64

3×3 64 → 64

1×1 64 → 256

69 632 tham số

hai conv 3×3 ở 256 kênh

3×3 256 → 256

3×3 256 → 256

1 179 648 tham số

ít hơn

≈ 17 lần

Bottleneck của ResNet-50 (layer1): conv 1×1 thu 256 kênh về 64, conv 3×3 chạy ở 64 kênh, conv 1×1 mở lại 256; 69 632 tham số so với 1 179 648 của hai conv 3×3 ở 256 kênh.

HỎI NHANH

`Conv2d(64, 128, 3)` đứng ngay trước `BatchNorm2d(128)`. Để `bias=True` thì thêm bao nhiêu tham số, và chúng có tác dụng gì không?

A 128, nhưng không có tác dụng, vì BatchNorm trừ đi mean của từng kênh

B 128, và có tác dụng giúp BatchNorm hội tụ nhanh hơn

C 8192, nhưng không có tác dụng, vì BatchNorm trừ đi mean của từng kênh

D 8192, và có tác dụng giúp BatchNorm hội tụ nhanh hơn

Depthwise separable

Depthwise: conv , `groups` bằng số kênh vào, mỗi bộ lọc đọc một kênh.

Pointwise: conv trộn kênh.

Nhỏ hơn gần lần.

HỎI NHANH

`nn.Conv2d(64, 64, 3, groups=64, bias=False).weight.shape` là gì?

A `(64, 64, 3, 3)`

B `(64, 1, 3, 3)`

C `(1, 64, 3, 3)`

D `(64, 3, 3)`

Residual giúp tối ưu, không chống học thuộc

ResNet-50 hơn 25 triệu tham số dễ học thuộc vài trăm ảnh.

Pretrained chỉ dùng khi đề cho phép; luôn so với baseline trên cùng split.

HỎI NHANH

Có 300 ảnh, đề cấm dùng model pretrained. ResNet-50 train từ đầu đạt 100% trên train, 55% trên validation. Một CNN nhỏ đạt 85% trên train, 78% trên validation. Bước tiếp theo hợp lý là gì?

A Giữ mạng nhỏ, thêm augmentation và regularization

B Thêm residual vào ResNet-50 cho dễ tối ưu hơn nữa

C Chọn ResNet-50, vì accuracy trên train cao hơn

D Lên ResNet-101 để học được đặc trưng sâu hơn

Bẫy: ReLU tại chỗ ở đầu nhánh

```
self.branch = nn.Sequential(nn.ReLU(inplace=True), conv)
return x + self.branch(x)
```

`inplace=True` ghi đè chính `x`: đường tắt cộng thay cho , không báo lỗi.

HỎI NHANH

Mọi tham số conv của block ở slide trên bằng 0. Đầu vào một kênh `[[-1., 2.]]` cho ra gì?

A `[[-1., 2.]]`

B `[[0., 0.]]`

C `[[-2., 4.]]`

D `[[0., 2.]]`

Khi đi thi

- Chạy một batch: in shape từng stage, đếm tham số.
- Baseline nhỏ có checkpoint trước, mạng sâu sau.
- Loss đứng ở mức đoán đều: xem gradient conv đầu.
- Đọc luật pretrained; ảnh nhỏ cần stem nhẹ.
- Đo thời gian trên đúng cấu hình nộp.

06

Luyện đề

LUYỆN ĐỀ 1

Bảng cấu hình ResNet-152 có conv , BatchNorm, pooling và phép cộng đường tắt. Thành phần nào là đóng góp riêng của ResNet, giúp train được mạng hơn 100 lớp?

A Đường tắt cộng đầu vào vào đầu ra của mỗi block

B BatchNorm đặt sau mỗi convolution

C Bộ lọc thay cho trong block

D Nhiều lớp pooling hơn giữa các stage

ĐỀ THI NOAI 2026 Singapore · Vòng sơ loại · Câu 216

LUYỆN ĐỀ 2

Một đội train hai CNN thường, không có đường tắt, cùng dữ liệu, cùng BatchNorm và khởi tạo He. Mạng 20 lớp có lỗi 6% trên train, 9% trên validation. Mạng 56 lớp có lỗi 11% trên train, 14% trên validation. Chẩn đoán và cách sửa nào hợp lý?

A Mạng sâu overfit, nên thêm dropout cho mạng 56 lớp

B Gradient quá lớn, nên clip độ lớn gradient về 1

C Mạng sâu khó tối ưu, nên thêm residual connection

D Mạng 56 lớp thừa tham số, nên giảm số kênh

PHỎNG THEO NOAI 2025 Singapore · Phần 1 · Câu 9

LUYỆN ĐỀ 3

Thay một conv thường, từ 64 lên 128 kênh, bằng depthwise separable: depthwise trên từng kênh, tức `groups=64`, rồi pointwise lên 128 kênh. Mọi conv đều không có bias. Số tham số trước và sau là bao nhiêu?

A và

B và

C và

D và

PHỎNG THEO NOAI 2025 Singapore · Phần 1 · Câu 14

Chốt ba điều

CNN: conv + ReLU + pooling + head; global average pooling xóa vị trí.

Residual giữ đường cho gradient; BatchNorm giữ thang tín hiệu.

ResNet: stage, projection, bottleneck, conv không bias trước BatchNorm.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook; đổi `blocks` trong thí nghiệm 18 lớp, dự đoán trước hàng nào kẹt, rồi kiểm lại bằng output.